

# claude-windows / ac48091f-a4f4-4268-b968-75205d9a50ff

## Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ac48091f-a4f4-4268-b968-75205d9a50ff\subagents\workflows\wf_b8ec0894-3cb\agent-ae384b5ad5237f400.jsonl`  
- SHA-256: `90108119f6dc3f11af407e7706925ae8db613996e36382beab5f3e5a98870d88`  
- Source modified: `2026-06-21T19:13:40+00:00`  
- Imported at: `2026-07-08T15:59:55+00:00`  
- project: `wf_b8ec0894-3cb`  
- session_id: `ac48091f-a4f4-4268-b968-75205d9a50ff`
```

## Transcript

### 1. user (2026-06-21T19:12:53.047Z)

You are verifying ONE claim in khelsutra's deploy runbook (deploy/DEPLOY\_ALPHA.md) against the REAL code the owner will deploy.

CRITICAL repo rules:

```
- Engine repo is at /c/Users/avidu/Projects/badminton-highlight-indexer. Its working tree is checked out on a STALE concurrent-session branch (12 commits behind master). DO NOT read the working tree for engine claims and DO NOT checkout/commit/modify anything there ? it is a shared checkout owned by another session.  
- To read what the owner actually deploys, read ENGINE origin/master ONLY, via:  
  git -C /c/Users/avidu/Projects/badminton-highlight-indexer grep -n "PATTERN" origin/master  
-- 'PATHGLOB'          (search master's tree)  
  git -C /c/Users/avidu/Projects/badminton-highlight-indexer show origin/master:PATH  
(read a master file)  
  git -C /c/Users/avidu/Projects/badminton-highlight-indexer log origin/master --oneline -i  
--grep="..."        (history)  
  (I already ran 'git fetch' so origin/master is current. Never fetch/checkout.)  
- khelsutra repo is at /c/Users/avidu/Projects/khelsutra, working tree is on clean master ? you MAY use Read/Grep/Glob directly there.
```

Be adversarial and literal: confirm the claim is TRUE for origin/master, with file:line evidence. If the doc says X but the code says Y, that is a 'discrepancy' ? capture it precisely. If something the owner must DO (beyond the doc's text) is implied, put it in owner\_action\_note. Verdict 'confirmed' only with concrete evidence; 'partial' if mostly true with a caveat; 'refuted' if wrong; 'unknown' if you genuinely could not determine it.

----

```
CLAIM (§5): `deploy/cloudflared.config.example.yml` defines the ingress (only api.khelsutra.guru ? http://127.0.0.1:8000) and is correct/complete for `cloudflared tunnel run`. This file is in the khelsutra repo: Read /c/Users/avidu/Projects/khelsutra/deploy/cloudflared.config.example.yml. Verify it: has a tunnel/credentials-file placeholder, an ingress rule api.khelsutra.guru ? http://127.0.0.1:8000, and the required catch-all `service: http_status:404` last rule (cloudflared REQUIRES a final catch-all or it errors). Flag anything missing or any value that contradicts §5's commands (tunnel name 'khelsutra-studio').
```

### 2. assistant (2026-06-21T19:12:56.360Z)

I'll verify the claim by reading the cloudflared config file in the khelsutra repo and cross-checking against §5 of the deploy runbook.

### 3. user (2026-06-21T19:12:56.896Z)

```
1      # cloudflared tunnel config ? KhelSutra Studio ENGINE (api.khelsutra.guru ? FastAPI on the box).
```

```

2      #
3      # Copy to the engine host (e.g. /etc/cloudflared/config.yml or
~/.cloudflared/config.yml), fill in
4      # <TUNNEL_UUID>, then run cloudflared as a service. The tunnel dials OUT to Cloudflare,
so NO inbound
5      # port is opened on the box ? the engine stays bound to 127.0.0.1:8000 (its safe
default), and the
6      # only way in is through Cloudflare Access. See DEPLOY_ALPHA.md.
7      #
8      # The SPA (app.khelsutra.guru) is NOT served here ? it ships on Cloudflare Pages
(DEPLOY_ALPHA.md §3).
9      # This tunnel carries ONLY the engine API.
10
11     tunnel: <TUNNEL_UUID>
12     credentials-file: /etc/cloudflared/<TUNNEL_UUID>.json
13
14     ingress:
15     # The engine FastAPI control plane (POST /api/process, GET /api/jobs, /api/query,
/api/compile,
16     # GET /api/highlights, /api/health). Bound to 127.0.0.1:8000 on the box ? never
0.0.0.0.
17     - hostname: api.khelsutra.guru
18       service: http://127.0.0.1:8000
19       originRequest:
20       # The async pipeline (fetch ? segment ? stitch) can run for minutes; don't let the
tunnel
21       # time the request out before the job is submitted (submit returns 202 fast, but
be generous).
22       connectTimeout: 30s
23       noTLSVerify: false
24
25     # A cloudflared ingress MUST end with a catch-all.
26     - service: http_status:404
27

```

#### 4. user (2026-06-21T19:12:57.714Z)

deploy\DEPLOY\_ALPHA.md

#### 5. assistant (2026-06-21T19:13:00.068Z)

Now let me verify §5 of the deploy runbook to cross-check the tunnel name and commands referenced.

#### 6. user (2026-06-21T19:13:00.477Z)

```

1      # Deploy KhelSutra Studio for alpha ? a shareable page, no SSH
2
3      > **Status:** `READY` (owner executes the cloud steps) · **Topology:** split ? SPA on
Cloudflare Pages + engine on an `api.*` cloudflared tunnel, Cloudflare Access over both (the
engine's `HOSTING_PLAN.md` §2 alpha plan). · **Cost:** ~$0/mo standing (Pages + Tunnel + Access
are free ?50 users) + Gemini per-run (~$0.05/match).
4
5      This turns the local-only flow (run engine + SPA on your box, drive it yourself) into a
**page you can share with alpha testers** ? they open `app.khelsutra.guru`, sign in (email
allowlist), point at a video, and get a reel. No SSH, no home IP exposed, no inbound ports.
6
7      ## 0. The shape
8
9      ...
10     tester browser
11     ? https
12     ?
13     Cloudflare edge (free): DNS · TLS · Access (email-OTP allowlist) in front of BOTH

```

```

hostnames
14      ??? app.khelsutra.guru ? Cloudflare Pages          (the web/ SPA; built with
VITE_API_BASE)
15      ??? api.khelsutra.guru ? cloudflared Tunnel (outbound-only) ?? 127.0.0.1:8000 on
YOUR box
16                                          ? FastAPI engine
17                                          ? (Gemini
segmenter; GPU optional)
18                                          ? async jobs,
SQLite, output/ on disk
19      ```
20
21      - **No inbound ports.** `cloudflared` dials out; the engine stays bound to
`127.0.0.1:8000`. The box's home/public IP is never exposed.
22      - **Auth at the edge.** Cloudflare Access gates both hostnames; the engine *also*
verifies the `Cf-Access` JWT in-app (defence in depth ? `security.edge_auth_enabled`), so a
direct hit to the tunnel can't spoof identity.
23      - **Cross-origin, on purpose.** `app.` (SPA) and `api.` (engine) are different origins ?
the SPA is built with `VITE_API_BASE=https://api.khelsutra.guru/api` (PR #64) and the engine
locks CORS to the SPA origin via `RALLY_CORS_ALLOW_ORIGINS` (no engine code change ? it's
already env-driven).
24
25      > **Alternative (single-origin):** if you'd rather run **one box** that serves the SPA
*and* the API behind one origin (Caddy reverse-proxy, no Pages, no cross-origin), see
`LOCAL_APPLIANCE_ARCHITECTURE.md` D1. This runbook ships the **split** topology you chose
(stable Pages-hosted page + a thin API tunnel).
26
27      ---
28
29      ## 1. Prerequisites
30
31      - A **box** to run the engine: the CPU droplet `168.144.126.176` (Gemini-only ? no GPU;
fine for alpha) **or** your Windows+WSL GPU box (real WASB). `ffmpeg` + Python 3.8+ installed;
the `badminton-highlight-indexer` engine checked out.
32      - A **Cloudflare account** with the `khelsutra.guru` zone (you already run the marketing
site there).
33      - `cloudflared` installed on the box (`https://developers.cloudflare.com/cloudflare-
one/connections/connect-networks/downloads/`).
34      - A **fresh** `GEMINI_API_KEY` (rotate the chat-shared one ? it uploads downsized chunks
to Google; this is a paid + cloud action).
35
36      ---
37
38      ## 2. Engine (the `api.` side)
39
40      ### 2.1 Install (with the auth extra)
41      ```bash
42      cd badminton-highlight-indexer
43      pip install -e .[auth]          # PyJWT[crypto] ? REQUIRED when edge auth is on, else
every request 500s
44      ```
45
46      ### 2.2 The safe-exposed config
47      Create / edit `config.json` on the box:
48      ```jsonc
49      {
50      "security": {
51          "edge_auth_enabled": true,                // turn the CF-Access JWT verify ON
52          "allowed_video_root": "/srv/khelsutra/incoming", // the path-jail ? REQUIRED when
exposed
53          "upload_dir": "/srv/khelsutra/incoming/uploads", // MUST sit UNDER
allowed_video_root (see §7)
54          "cf_team_domain": "https://<your-team>.cloudflareaccess.com",
55          "cf_access_aud": "<the CF Access application AUD>" // filled in §4
56      }

```

```

57     }
58     ```
59     And export the env (a `.env` or your shell):
60     ```bash
61     export GEMINI_API_KEY="<your-rotated-key>"
62     export RALLY_CORS_ALLOW_ORIGINS="https://app.khelsutra.guru"    # lock CORS to the SPA
origin
63     ```
64
65     > **The engine refuses to start unsafe.** The boot posture (engine PR #294) **fails fast** if `edge_auth_enabled` is on but the `allowed_video_root` jail or
`cf_team_domain`/`cf_access_aud` is missing, and **warns** if `PyJWT` isn't installed. So a
half-configured exposure can't silently come up ? fix what it prints and restart.
66
67     ### 2.3 Run
68     ```bash
69     python -m backend.main --server      # binds 127.0.0.1:8000 (the safe default ? do NOT
change the bind)
70     ```
71     Keep it alive with a process manager (`systemd` on the droplet, mirroring
`site/scripts/provision-vm.sh`'s unit; or `pm2`/`nssm` on Windows).
72
73     ---
74
75     ## 3. The SPA on Cloudflare Pages (the `app.` side)
76
77     The SPA ships as a **static Pages site**, git-connected like the marketing site (auto-
deploy on merge to `master`).
78
79     1. **Cloudflare dashboard ? Workers & Pages ? Create ? Pages ? Connect to Git** ? pick
the `khelsutra` repo.
80     2. **Build settings:**
81         - **Root directory:** `web`
82         - **Build command:** `npm run build`
83         - **Build output directory:** `web/dist`
84         - **Environment variable:** `VITE_API_BASE = https://api.khelsutra.guru/api` ? this
is what points the SPA at the engine tunnel (PR #64).
85     3. **Custom domain:** add `app.khelsutra.guru` to the Pages project.
86
87     > Every push to `master` now rebuilds + redeploys the SPA. To preview a change first,
Pages builds a per-PR preview URL.
88
89     ---
90
91     ## 4. Cloudflare Access (gate BOTH hostnames)
92
93     1. **Zero Trust ? Access ? Applications ? Add ? Self-hosted.**
94     2. **Application domain:** add **both** `app.khelsutra.guru` **and**
`api.khelsutra.guru` to the *same* application (one app ? one shared session cookie, so a tester
signs in once and both the SPA and its API calls are authorized).
95     3. **Policy:** Action *Allow*, rule *Emails* ? your alpha testers' addresses (free ? 50
users ? fits 10?25 testers). Use one-time-PIN (email OTP) so testers need no Cloudflare account.
96     4. **Copy the application `AUD` tag** ? paste into the engine's `config.json`
`security.cf_access_aud` ($2.2), and set `cf_team_domain` to `https://<your-
team>.cloudflareaccess.com`. Restart the engine.
97     5. **CORS preflight ?:** the browser sends an unauthenticated `OPTIONS` preflight to
`api.` for the cross-origin POSTs. In the Access application's **Settings ? enable ?Bypass
options requests to CORS preflight?* (or add an explicit OPTIONS bypass), else the preflight is
redirected to the login page and every API call fails. The engine's CORS middleware
(`allow_credentials=false`, no cookies) then answers the preflight.
98
99     ---
100
101     ## 5. The tunnel (engine ? Cloudflare)
102

```

```

103     ``bash
104     cloudflared tunnel login                                # browser auth to your zone
105     cloudflared tunnel create khelsutra-studio              # note the <TUNNEL_UUID>
106     # copy deploy/cloudflared.config.example.yml ? /etc/cloudflared/config.yml, fill
<TUNNEL_UUID>
107     cloudflared tunnel route dns khelsutra-studio api.khelsutra.guru
108     cloudflared tunnel run khelsutra-studio                  # or install as a service:
`cloudflared service install`
109     ``
110     See `deploy/cloudflared.config.example.yml` for the ingress (only `api.khelsutra.guru ?
http://127.0.0.1:8000`).
111
112     ---
113
114     ## 6. Verify (the posture test + the CUJ)
115
116     **Exposure posture (must hold ? this is what makes it safe to share):**
117     - From OUTSIDE the box: `curl --max-time 5 http://<box-public-ip>:8000/api/health` ?
**refused / timeout** (the engine is `127.0.0.1`-bound; no inbound port). ? The box is not
directly reachable.
118     - `curl -i https://api.khelsutra.guru/api/health` **without** a CF session ? a **302 to
the Access login** (not a 200). ? The gate is in front.
119     - Open `https://app.khelsutra.guru` in a browser ? CF Access login ? after sign-in, the
SPA loads and `/api/health` returns 200 (same session cookie). ?
120
121     **The shareable CUJ (what a tester does):** open `app.khelsutra.guru` ? sign in ? paste
a `gs://` video (or a path under `allowed_video_root`) ? **Process** ? watch progress ? pick
rallies ? **Build** ? **download the reel**. All from the page.
122
123     ---
124
125     ## 7. Honest caveats / deferred
126
127     - **`upload_dir` must sit UNDER `allowed_video_root`.** If you enable browser upload, an
upload landing outside the jail is rejected by `/api/process` (a functional break, not a hole).
Keep `upload_dir` inside the jail (§2.2).
128     - **You MUST set `edge_auth_enabled=true` to expose.** The boot posture (and the in-app
JWT verify) only engage then. Do not put the engine behind the tunnel with edge auth off.
129     - **Cross-origin reel download:** the reel streams from `api.` with `Content-
Disposition: attachment`, so it downloads even cross-origin ? but verify the filename in your
browser during the CUJ (cross-origin `<a download>` filename hints are ignored; the header is
the authority).
130     - **Upload size:** Cloudflare's free proxy caps request bodies at **100 MB** ? large
matches upload as 90 MB chunks (the SPA chunked-upload path), or pre-stage to a bucket and
ingest the `gs://` URI (already supported).
131     - **Box offline = engine offline.** The Pages SPA stays up and shows a friendly "engine
offline" banner (`/api/health` ping); processing resumes when the box is back. Beta removes this
with a queue (`HOSTING_PLAN.md` §3).
132     - **Gemini = paid + uploads footage to Google.** Keep the key rotated; use it only for
clips you're OK sending.
133
134     ---
135
136     ## 8. Teardown
137     `cloudflared tunnel cleanup khelsutra-studio` + delete the tunnel; remove the Pages
custom domain; disable the Access application; stop the engine service. The box's `output/` +
`uploads/` retain originals ? sweep them per your retention policy before any wider exposure.
138
139     ---
140     ### Changelog
141     - 2026-06-21 ? created. Split-topology alpha deploy kit (cloudflared + Pages + CF
Access) consuming the engine exposure-safety boot posture (engine #294) + the SPA
`VITE_API_BASE` (#64). Pairs with `LOCAL_APPLIANCE_ARCHITECTURE.md` (single-origin alternative)
+ the engine `HOSTING_PLAN.md` §2.
142

```

## 7. assistant (2026-06-21T19:13:16.291Z)

I have everything I need to verify the claim. Let me analyze the cloudflared config against \$5's requirements.

The claim has several testable parts. Let me check each:

1. **\*\*Tunnel/credentials-file placeholder\*\*** ? Lines 11-12: ``tunnel: <TUNNEL_UUID>`` and ``credentials-file: /etc/cloudflared/<TUNNEL_UUID>.json``. Present. ?
2. **\*\*Ingress rule `api.khelsutra.guru` ? http://127.0.0.1:8000`\*\*** ? Lines 17-18: ``- hostname: api.khelsutra.guru`` / ``service: http://127.0.0.1:8000``. Present and exact. ?
3. **\*\*Required catch-all `- service: http\_status:404` last rule\*\*** ? Line 26: ``- service: http_status:404``, and it IS the last ingress rule. ?
4. **\*\*Tunnel name consistency\*\*** ? \$5 uses ``cloudflared tunnel run khelsutra-studio`` (line 108) and references ``<TUNNEL_UUID>``. The config uses the UUID for both ``tunnel:`` and ``credentials-file:``, not the name. This is a notable nuance worth flagging.

Now there's an important adversarial point about how ``cloudflared tunnel run`` interacts with the config file. Let me check the cloudflared semantics carefully.

The config at line 11 sets ``tunnel: <TUNNEL_UUID>``. The \$5 command is ``cloudflared tunnel run khelsutra-studio`` (passing the tunnel name as an arg). In cloudflared, when you pass a tunnel name/UUID on the CLI to ``tunnel run``, it must match (or the CLI arg takes precedence / can conflict with) the ``tunnel:`` key in the config. If both are set and they refer to the same tunnel, it works. But there's a subtlety: ``cloudflared tunnel route dns khelsutra-studio`` and ``tunnel run khelsutra-studio`` use the **\*\*name\*\***, while the config file's ``tunnel:`` field has the **\*\*UUID\*\*** placeholder. These resolve to the same tunnel, so it's consistent ? but the operator must fill ``<TUNNEL_UUID>`` with the actual UUID from ``tunnel create`` (line 105 says "note the ``<TUNNEL_UUID>``").

Let me double check that the config path \$5 instructs matches where cloudflared looks. \$5 line 106 says copy to ``/etc/cloudflared/config.yml``, and ``tunnel run`` looks for config at ``/etc/cloudflared/config.yml`` (or ``~/.cloudflared/config.yml``) by default. The config header (lines 3-4) names exactly those paths. Consistent.

One genuinely adversarial check: does the ``originRequest`` block with ``connectTimeout``/``noTLSVerify`` cause any validation error? These are valid cloudflared ``originRequest`` keys, so no. The config is well-formed YAML.

Let me verify there's no contradiction I'm missing regarding the ``credentials-file`` path vs. where ``tunnel create`` writes credentials ? ``tunnel create`` writes to ``~/.cloudflared/<UUID>.json`` by default, but the config points at ``/etc/cloudflared/<UUID>.json``. The operator must copy/move the credentials JSON to that path (or adjust the path). This is an implied owner action.

The config is correct and complete for ``cloudflared tunnel run``. Let me capture the verdict.

## 8. user (2026-06-21T19:13:32.549Z)

Structured output provided successfully

## 9. assistant (2026-06-21T19:13:40.474Z)

Verified. The claim is **\*\*confirmed\*\***.

``/c/Users/avidu/Projects/khelsutra/deploy/cloudflared.config.example.yml`` is correct and complete for ``cloudflared tunnel run``:

- **\*\*Tunnel/credentials placeholders present\*\*** (lines 11-12): ``tunnel: <TUNNEL_UUID>`` + ``credentials-file: /etc/cloudflared/<TUNNEL_UUID>.json``.
- **\*\*Single ingress rule, exact match\*\*** (lines 17-18): ``api.khelsutra.guru`` ? ``http://127.0.0.1:8000``. It is the only hostname rule.

- **\*\*Required final catch-all present and last\*\*** (line 26): ``- service: http_status:404`` ? cloudflared errors without this, and it is correctly positioned as the last ingress entry.
- **\*\*No contradiction with \$5's `khelsutra-studio` name\*\***: \$5 keys CLI commands by tunnel name; the config keys by UUID ? both resolve to the same tunnel, and \$5 line 105 tells the operator to capture the UUID.

Two implied owner actions worth flagging (not doc errors, just operational gaps the placeholders assume): the operator must (1) substitute both ``<TUNNEL_UUID>`` occurrences with the real UUID, and (2) move the credentials JSON that ``tunnel create`` writes to ``~/.cloudflared/<UUID>.json`` into ``/etc/cloudflared/<UUID>.json`` to match the ``credentials-file`` path, or ``tunnel run`` won't find credentials. Severity: none.